

École de technologie supérieure

Département de génie logiciel et des TI

Rapport de Laboratoire 2

Filtrage spatial et Détection de Panneaux de Signalisation

Cours : GTI411 – Imagerie numérique

Chargée de laboratoire : Alice Le Nilias Houmeau

Nom : JONNIAUX

Prénom : Mathieu

Automne 2026

Table des matières

1 Introduction **2**

2 Outils et concepts **2**

2.1 Concepts fondamentaux 2

3 Expérimentations et discussion **3**

3.1 Partie 1 : Sélection manuelle des zones 3

3.1.1 Logique de l'algorithme de similarité 3

3.1.2 Justification des étapes de traitement 3

3.1.3 Résultats de la Partie 1 6

3.2 Partie 2 : Détection automatisée des zones 8

3.2.1 Logique de détection automatique 8

3.2.2 Justification des étapes du pipeline 8

3.2.3 Résultats sur les images de test 10

4 Difficultés et améliorations **11**

4.1 Difficultés rencontrées et limites 11

4.2 Améliorations proposées (sans machine learning) 12

5 Conclusion **12**

1 Introduction

Dans le cadre du cours GTI411, ce deuxième laboratoire a pour but d'appliquer le filtrage, les espaces de couleurs et le calcul de similarité entre images à un cas concret : retrouver des panneaux de signalisation dans des photos. Comme demandé, je n'ai utilisé que des méthodes de traitement d'image classiques vues en cours, aucun modèle d'apprentissage profond.

Le travail se divise en deux parties :

1. **Partie 1 (détection manuelle)** : l'utilisateur dessine un rectangle à la souris sur l'image. Le programme compare cette zone aux 14 images de référence (*templates*) et affiche le panneau le plus ressemblant avec son score, ou « Inconnu » si rien ne ressemble assez.
2. **Partie 2 (détection automatisée)** : le programme trouve lui-même les zones qui ressemblent à des panneaux, enlève les doublons, puis réutilise la comparaison de la partie 1 sur chacune d'elles.

Le code est organisé en MVC (modèles, vues, contrôleurs) en Python. Ce rapport présente d'abord les concepts utilisés, puis la logique et les résultats de chaque partie, et enfin les difficultés rencontrées et les améliorations possibles.

2 Outils et concepts

2.1 Concepts fondamentaux

Voici les notions du cours que j'ai utilisées :

— **OpenCV pour la lecture et l'affichage** : chargement des images, rectangle tracé à la souris, dessin des boîtes et du texte.

— **Filtre gaussien** : je l'utilise avant presque tout (seuillage, comparaison) pour enlever le bruit.

— **Gradient de Sobel** : deux noyaux 3×3 donnent g_x et g_y, et la norme $g = \sqrt{g_x^2 + g_y^2}$ fait ressortir les contours. C'est la base de ma comparaison, parce qu'un contour reste le même qu'il fasse sombre ou clair.

— **Laplacien** : je m'en sers seulement pour mesurer la netteté (sa variance est faible sur une zone floue).

— **HSV, seuillage et morphologie** : `inRange` sur la teinte pour les panneaux colorés, un seuil sur S et V pour le blanc, puis une fermeture et une ouverture pour nettoyer les masques. Ensuite, `findContours` et `boundingRect` pour avoir les boîtes.

— **Corrélation normalisée** (`matchTemplate`, `TM_CCOEFF_NORMED`) : elle enlève la moyenne et normalise, donc la luminosité globale compte peu.

Les bibliothèques utilisées sont OpenCV (OpenCV, 2024) pour les filtres, les conversions et la corrélation, et NumPy (NumPy, 2026) pour les calculs sur les matrices d'images.

3 Expérimentations et discussion

3.1 Partie 1 : Sélection manuelle des zones

3.1.1 Logique de l'algorithme de similarité

Au début, je pensais simplement comparer la zone sélectionnée à chaque template pixel par pixel. Ça marche très mal : les templates sont des dessins propres sur fond blanc, alors que la zone choisie à la souris est une vraie photo, avec des ombres, une échelle différente et souvent un bout d'arrière-plan (arbres, sable, route) dans les coins du rectangle. J'ai donc combiné les contours (Sobel), un balayage de plusieurs échelles et une vérification des couleurs en HSV. L'ensemble est résumé à la Figure 2.

3.1.2 Justification des étapes de traitement

— **Gradient de Sobel** : en comparant les contours (voir Figure 1) plutôt que les niveaux de gris, le fond blanc du template et le fond naturel de la photo comptent beaucoup moins, et un panneau trop clair ou trop sombre garde les mêmes arêtes que son modèle.

— **Flou gaussien** 5×5 : je l'applique sur la carte de Sobel et sur la version couleur. Ça adoucit les arêtes et la corrélation supporte mieux un petit décalage entre la photo et le dessin.

— **Sept échelles (70% à 130%)** : le rectangle tracé à la main est rarement ajusté au panneau. Tester plusieurs tailles permet de trouver la bonne correspondance malgré la marge.

— **Couleurs HSV** : pour chaque image, je calcule la part de pixels rouges, oranges, jaunes, verts, bleus, noirs et blancs. Pour un template coloré, je regarde quelle proportion de ses couleurs vives se retrouve dans la zone, donc un template rouge ne peut pas être accepté pour une zone sans rouge. Pour les panneaux noir et blanc, je pénalise les zones très colorées. Le blanc et le noir ne comptent pas dans la comparaison, sinon tous les panneaux noir et blanc se ressemblaient.

— **Pondération 70% / 30%** : les contours sont plus fiables, la couleur sert surtout à confirmer.

— **Rapport largeur/hauteur** : si les proportions de la zone et celles du template diffèrent de plus de 40%, je divise le score par 2. Une sélection très allongée ne peut donc plus être associée à un panneau carré ou rond.

— **Seuil de 0.33** : choisi en testant. Plus bas, des zones de décor étaient étiquetées,

plus haut, des panneaux un peu déformés étaient perdus.



FIGURE 1 – Carte d'amplitude du gradient de Sobel extraite sur le template STOP Crédit : JONNIAUX Mathieu

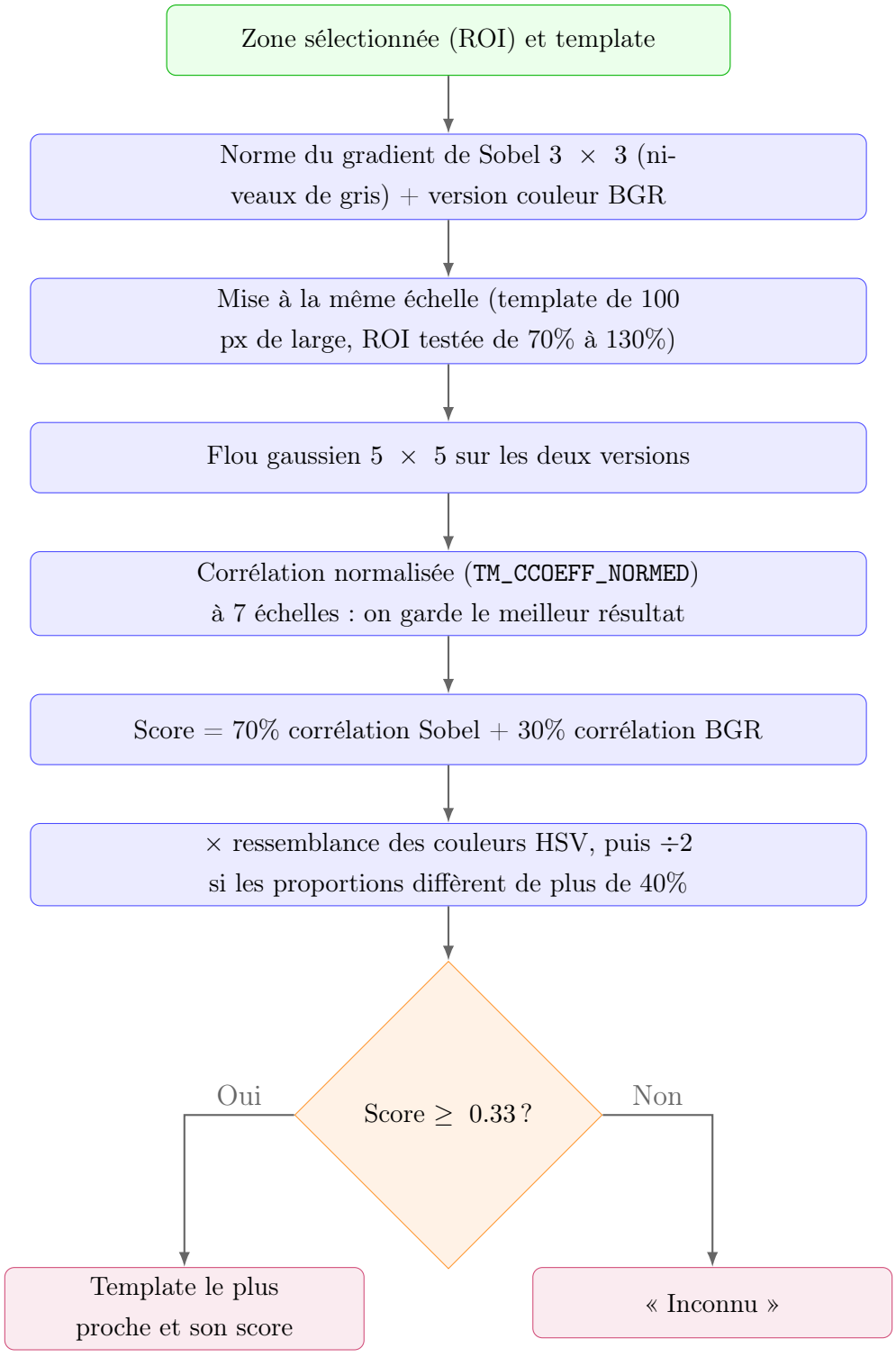


FIGURE 2 – Diagramme de flux de l'algorithme de similarité de la Partie 1 Crédit : JONNIAUX Mathieu

3.1.3 Résultats de la Partie 1

J'ai sélectionné les panneaux de l'image 1 un par un, avec des rectangles volontairement un peu trop grands ou décalés. La Figure 3 montre le panneau STOP sélectionné avec de la marge : il est reconnu avec un score de 0.75. Sur des zones de sable ou de route, le score reste bas (0.13 et 0.14) et sur une zone d'arbres il atteint 0.16 : dans les trois cas, le programme affiche « Inconnu » (Figure 4), bien en dessous du seuil de 0.33. J'ai aussi testé l'image 2 (Figure 5) : le panneau DEER XING est reconnu avec un score de 0.81.



FIGURE 3 – Sélection manuelle et résultat de similarité sur l'image 1 Crédit : JONNIAUX Mathieu

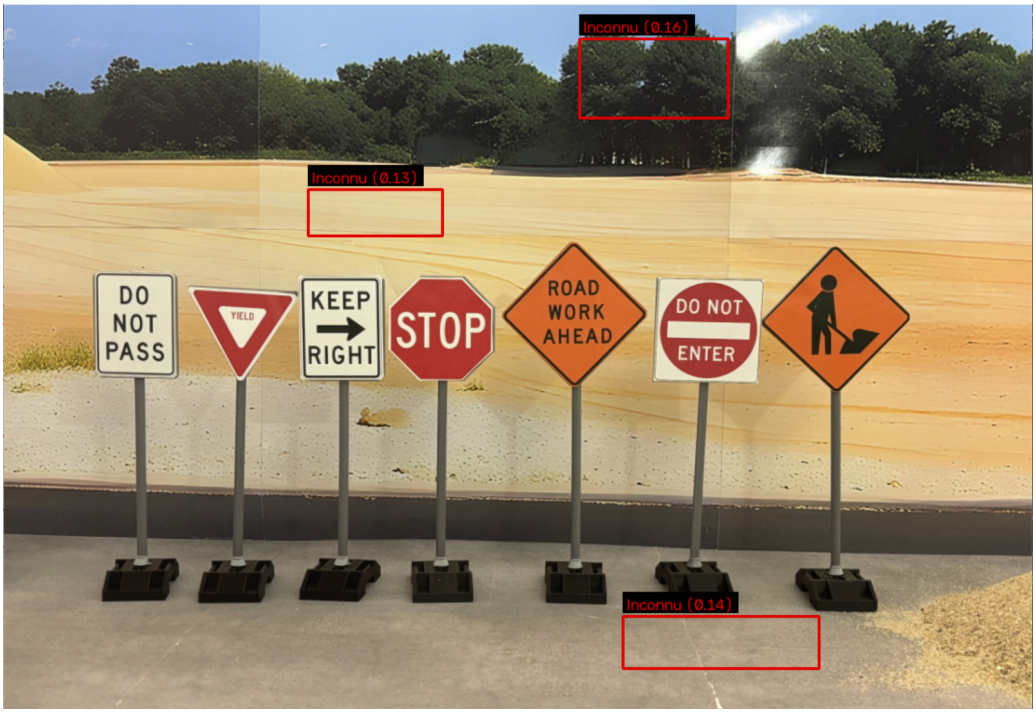


FIGURE 4 – Sélections sur le sable, la route et les arbres : résultat « Inconnu » Crédit : JONNIAUX Mathieu



FIGURE 5 – Sélection manuelle et résultat de similarité sur l'image 2 Crédit : JONNIAUX Mathieu

3.2 Partie 2 : Détection automatisée des zones

3.2.1 Logique de détection automatique

Ici, le programme remplace l'utilisateur : il cherche seul les zones candidates, puis applique l'algorithme de la partie 1. Les étapes sont décrites à la Figure 7.

3.2.2 Justification des étapes du pipeline

— **Un masque par couleur** : au début, j'avais combiné tous les masques en un seul. Sur l'image 2, les panneaux de la rangée du bas formaient alors un seul gros bloc, rejeté parce que trop grand. En gardant les masques séparés avant `findContours`, chaque panneau reste un objet distinct.

— **Fermeture puis ouverture** : le texte et les symboles noirs font des trous dans le masque. La fermeture comble la plupart de ces trous, et l'ouverture enlève les petites taches du fond (voir Figure 6). La taille du noyau dépend de la taille de l'image (0.8% du plus petit côté).

— **Filtres géométriques en proportion de l'image** : rien n'est fixé en pixels, parce que les images de l'évaluation auront d'autres résolutions.

— **Fusion** : la bordure rouge et l'intérieur blanc d'un même panneau donnent deux boîtes, que je fusionne. Je ne fusionne que des boîtes de taille comparable (rapport d'aire ≤ 3), sinon une grande tache claire du fond avale le panneau qu'elle contient.

— **Netteté par le Laplacien** : la voiture rouge floue de l'image 2 passe les filtres de couleur et de taille. En mesurant la variance du Laplacien sur la zone ramenée à 100×100 pixels, j'ai obtenu entre 6 et 26 pour la voiture et la route, contre plus de 1000 pour tous les vrais panneaux, d'où le seuil de 150.

— **Doublons** : si une grande boîte et une boîte mieux ajustée contiennent le même panneau, je garde celle qui a le meilleur score. Si un même template sort plusieurs fois, je garde seulement les détections qui ont au moins 50% du meilleur score, ce qui enlève des fragments d'autres panneaux.



FIGURE 6 – Masque rouge après morphologie sur l'image 1 (quelques petits trous restent)
Crédit : JONNIAUX Mathieu

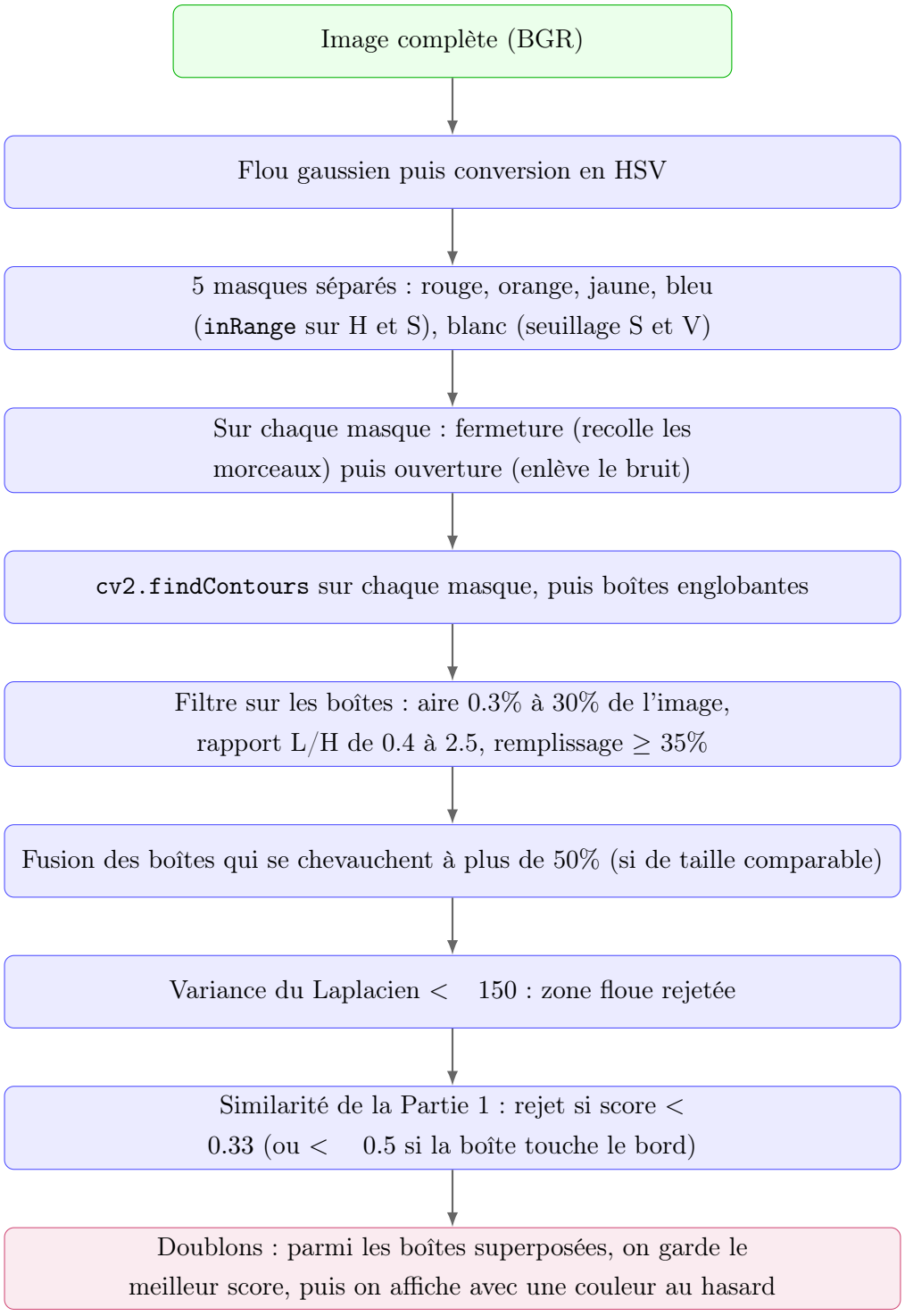


FIGURE 7 – Diagramme de flux de la détection automatique (Partie 2) Crédit : JONNIAUX Mathieu

3.2.3 Résultats sur les images de test

Image 1 (`traffic_signs_1.jpg`) : les 7 panneaux sont trouvés et correctement identifiés, sans fausse détection (Figure 8) : `stop` (0.82), `road_work_ahead` (0.77), `dont_pass` (0.73), `dont_enter` (0.73), `work` (0.64), `keep_right` (0.58) et `yield` (0.52).



FIGURE 8 – Résultats de la détection automatique sur l'image 1 (7/7 panneaux) Crédit : JONNIAUX Mathieu

Image 2 (`traffic_signs_2.jpg`) : cette image est plus difficile (beaucoup de panneaux, fond flou, voiture rouge). Les 6 panneaux qui ont un template sont trouvés, sans fausse étiquette (Figure 9) : `dont_enter` (0.86), `deer_xing` (0.85), `40_sign` (0.68), `stop` (0.58), `not_turn` (0.50) et `traffic_lights` (0.38). Les autres panneaux (limite de vitesse, école, flèche bleue, etc.) n'ont pas de template, et ils ne sont pas étiquetés : ils sont rejetés par le filtre de netteté ou par le seuil de similarité. La voiture rouge est rejetée par le Laplacien.



FIGURE 9 – Résultats de la détection automatique sur l'image 2 (6/6 panneaux ayant un template) Crédit : JONNIAUX Mathieu

4 Difficultés et améliorations

4.1 Difficultés rencontrées et limites

— **La comparaison des templates** : mon premier essai (`matchTemplate` direct sur les niveaux de gris) donnait des scores très bas et beaucoup d'erreurs. J'ai ensuite essayé de comparer des histogrammes HSV avec les contours de Canny : environ 5 panneaux sur 7 étaient bons, mais `dont_pass` et `dont_enter` échouaient. J'ai aussi essayé d'ignorer le fond blanc des templates, mais ça a effacé l'intérieur des panneaux blancs (`dont_pass`, `keep_right`), et plus rien ne passait. Le passage au gradient de Sobel avec plusieurs échelles a réglé le problème.

— **Un score de couleur trop généreux** : en mesurant chaque composante du score, j'ai vu que le score de couleur valait presque 1 pour n'importe quel panneau noir et blanc, donc il ne servait à rien et créait de fausses étiquettes. Ne comparer que les couleurs vives a corrigé ça.

— **Les panneaux blancs** : trouver un seuil pour le blanc a été le plus long. En mesurant les valeurs HSV, le blanc crème de l'image 1 a une saturation de 31 à 42, alors que le sable est entre 47 et 90. Avec $S < 30$, je perdais `dont_pass` et `keep_right` ; j'ai donc pris $S < 48$ et $V > 200$. Avec un V trop élevé, `dont_enter` disparaissait de l'image 2 parce qu'une grande tache claire du fond avalait son panneau à la fusion, ce qui m'a amené à limiter la fusion aux boîtes de taille comparable.

— **Seuils réglés sur deux images** : j'ai choisi tous mes seuils en testant sur seulement deux images, donc rien ne garantit les mêmes résultats sur celles de l'évaluation. Le seuil de 0.33 est aussi un compromis, et le template le plus faible (`traffic_lights`, score de 0.38 sur l'image 2) en est proche. Enfin, la corrélation

ne gère pas les rotations importantes ni les fortes déformations de perspective, et le filtre de netteté rejetterait un vrai panneau flou.

4.2 Améliorations proposées (sans machine learning)

— **Forme du contour** : je calcule déjà le remplissage (aire du contour / aire de la boîte) pour filtrer les boîtes. On pourrait aussi s'en servir pour deviner la forme : environ 1 pour un panneau rectangulaire, 0.8 pour un rond, ect On ne comparerait alors qu'avec les templates de même forme, ce qui irait plus vite et éviterait des confusions entre panneaux de même couleur. (En cherchant sur internet, j'ai aussi vu qu'il existait `cv2.approxPolyDP` qui a aussi pour but de détecter la forme via les sommets)

— **Seuil de blanc adaptatif** : utiliser Otsu ou un seuil local au lieu de valeurs fixes, qui risquent d'échouer sur un autre fond clair.

— **Égalisation d'histogramme** sur le canal V avant le seuillage du blanc, pour moins dépendre de l'éclairage et mieux voir les panneaux à l'ombre.

5 Conclusion

Ce laboratoire m'a permis de combiner ce qu'on a vu en cours : espaces de couleurs pour trouver les zones candidates, morphologie et contours pour les isoler, et filtres (Sobel, Laplacien) pour les comparer et filtrer les faux positifs. J'ai surtout compris qu'une comparaison pixel par pixel ne suffit pas entre un dessin propre et une photo, et que comparer des contours à plusieurs échelles marche beaucoup mieux. Le programme final reconnaît les 7 panneaux de l'image 1 et les 6 panneaux de l'image 2 qui ont un template, sans fausse détection.

Références

OpenCV : *Open Source Computer Vision Library Documentation*, en ligne : https://docs.opencv.org/.

NumPy Developers : *NumPy Reference Manual*, en ligne : https://numpy.org/doc/.

Vázquez, C. (2026) : *Notes de cours GTI411 – Imagerie numérique*, École de technologie supérieure (ÉTS), Montréal.